



AI with Rav

Learn AI the way you actually think.



DAY 17 of 30

MACHINE LEARNING

Evaluation Metrics — accuracy isn't enough

WHAT YOU'LL LEARN TODAY

- › The accuracy trap (rare things)
- › The confusion matrix — 4 outcomes
- › Precision — avoiding false alarms
- › Recall — avoiding dangerous misses
- › F1 — balancing both



Evaluation Metrics — accuracy isn't enough

In One Line: A test for a rare disease can be "99% accurate" and still catch ZERO sick people. Accuracy lies when one class is rare. Today you learn the metrics that tell the real story — precision, recall, and F1.

Start here: when a high score is a lie

Your model proudly reports **"95% accuracy."** Sounds great, right? But here's the uncomfortable truth every ML engineer must learn: **a high accuracy can secretly hide a completely useless model.** Especially when the thing you're looking for is **rare** — like a disease, fraud, or spam.

This is the video that turns a beginner into someone you can trust with a real model. Because knowing your model *works* is not about one big number — it's about asking the *right* questions.

The accuracy trap

The accuracy trap: 99% "accurate" — and useless



1000 patients: only 10 are truly sick (red), 990 are healthy (green). A lazy model that calls EVERYONE "healthy" is right 990 out of 1000 = 99% accurate — yet it caught none of the 10 sick people. High accuracy, zero usefulness.

Imagine a disease test. Out of **1000 patients, only 10 are actually sick.** Now build the laziest possible model: it says **"healthy"** to everyone.

- It's right about all **990 healthy** people.
- So its accuracy = $990/1000 = 99\%$. Looks amazing!
- But it **caught ZERO of the 10 sick people.** A test that never finds the sick is worthless.



That's the accuracy trap. When one group is **rare**, a model can score high just by ignoring it. 99% accuracy, 0% useful. **Never judge a model by accuracy alone** — especially for rare, important events like disease, fraud, or spam. We need sharper questions.

The confusion matrix: 4 outcomes

The confusion matrix: 4 possible outcomes

		What the model SAID	
		"sick"	"healthy"
the TRUTH	really sick	TRUE POSITIVE sick + caught (correct)	FALSE POSITIVE healthy but alarmed (false alarm)
	really healthy	FALSE NEGATIVE sick but MISSED (dangerous!)	TRUE NEGATIVE healthy + cleared (correct)

A 2×2 grid. Down the side is the TRUTH (really sick / really healthy); across the top is what the model SAID. The four boxes: True Positive (caught it), False Positive (false alarm), False Negative (missed it — dangerous), True Negative (correctly cleared).

Every prediction lands in one of **four boxes**. This grid is called the **confusion matrix**, and it's the foundation of all the good metrics:

- **True Positive** → sick, and the model **caught it**. ✓ Correct.
- **False Positive** → healthy, but the model **cried "sick"** — a false alarm.
- **False Negative** → sick, but the model **MISSED it**. The dangerous one.
- **True Negative** → healthy, and the model **correctly cleared** them. ✓ Correct.



Read it as: **the truth on the side, the model's guess on top**. The two green boxes are the model being right; the two red boxes are its two *different* kinds of mistakes — a **false alarm** (annoying) and a **miss** (often dangerous). These four numbers tell the real story that accuracy hides.

Precision — avoiding false alarms

Precision: of everyone you CALLED sick, how many really were?



model called these 8 people "sick"

6 were really sick, 2 were false alarms

PRECISION = 6 / 8 = 75%

"When I say sick, I'm right 75% of the time" — avoids false alarms

Of the 8 people the model called "sick", 6 were truly sick and 2 were false alarms. Precision = $6/8 = 75\%$. It answers: when you shout "sick", how often are you right?

The first sharp question: **"When my model says sick, how often is it actually right?"** That's **precision**.

- The model called **8 people** "sick."
- **6** of them really were; **2** were false alarms.
- **Precision = 6 / 8 = 75%** → three out of four alarms are real.

Precision is about **avoiding false alarms**. High precision means "when I raise the flag, trust me." You want high precision when a false alarm is costly — like flagging a good customer's card as fraud and freezing it, or marking an important email as spam.

Recall — avoiding dangerous misses



Recall: of everyone truly sick, how many did you CATCH?



these 10 people were really sick

caught 6, MISSED 4

$$\text{RECALL} = 6 / 10 = 60\%$$

"I catch 60% of the truly sick" — avoids dangerous misses

Of the 10 people who were truly sick, the model caught 6 and MISSED 4. Recall = $6/10 = 60\%$. It answers: of everyone who really has it, how many did you catch?

The second sharp question: "Of everyone who is truly sick, how many did my model catch?" That's **recall**.

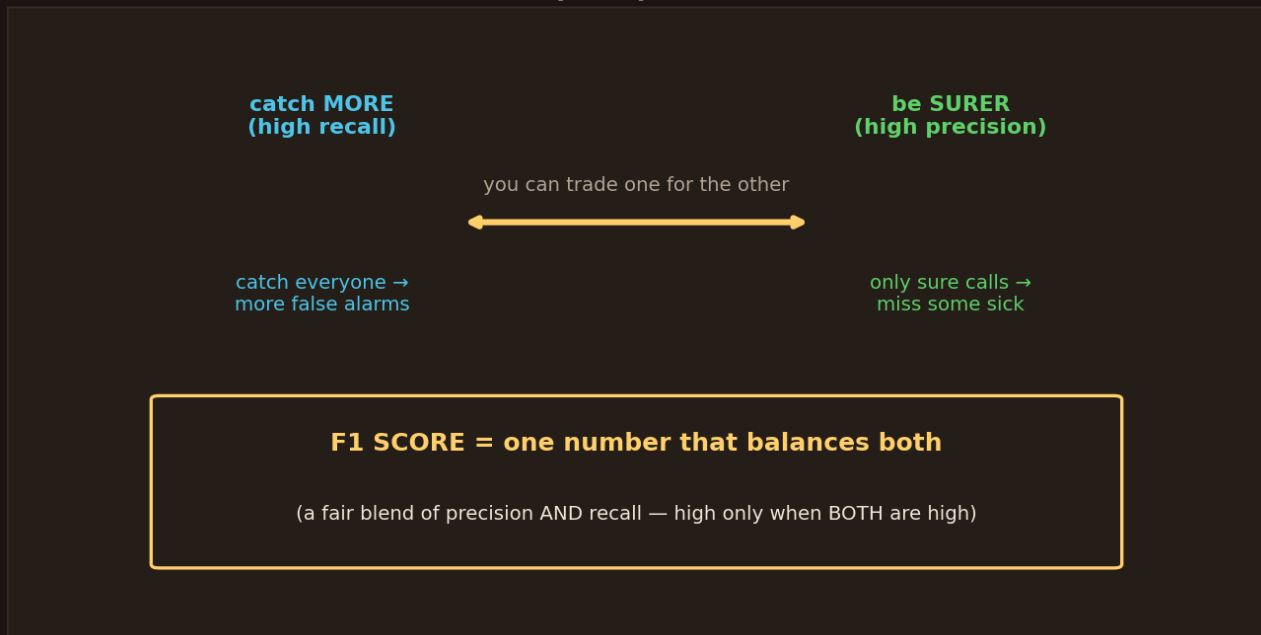
- There were **10 truly sick** people.
- The model **caught 6** and **missed 4**.
- **Recall = $6 / 10 = 60\%$** → it finds six of every ten sick people.

Recall is about **avoiding dangerous misses**. High recall means "I rarely let a real case slip through." You want high recall when a miss is dangerous — a missed cancer, an undetected fraud, a bomb in a bag. Better a few false alarms than one deadly miss.

The tradeoff — and F1



Precision vs Recall pull apart — F1 balances them



Precision and recall pull against each other. Catch more sick people (high recall) → more false alarms (lower precision). Only make sure calls (high precision) → miss some (lower recall). F1 is one number that balances both.

Here's the tension: **you usually can't max out both.**

- Want to **catch everyone** (high recall)? You'll flag lots of healthy people too → **more false alarms** (lower precision).
- Want to **only be sure** (high precision)? You'll skip the doubtful cases → **miss some sick** (lower recall).
- So you **tune the balance** based on what's worse for your problem: a miss or a false alarm.

When you want a single number that respects **both**, use the **F1 score** — a fair blend of precision and recall. The clever part: F1 is high **only when BOTH are high**. If either one is bad, F1 drops. It stops you from cheating by maxing one and ignoring the other.

See it in code

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score, f1_score

y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))  # the 4 boxes: [[TN, FP], [FN, TP]]
print("precision:", precision_score(y_test, y_pred))  # avoid false alarms
print("recall:   ", recall_score(y_test, y_pred))    # avoid misses
print("f1:       ", f1_score(y_test, y_pred))        # the balance
```



One line each. ``confusion_matrix`` shows all four outcomes; then read ``precision``, ``recall``, and ``f1``. For any rare-event problem (disease, fraud, spam), **report these — not accuracy**. This is the habit that separates real ML from a misleading demo.

Which one do YOU want?

- **Spam filter** → high **precision** (don't send a real, important email to junk — a false alarm is costly).
- **Cancer screening** → high **recall** (never miss a real case — a miss is deadly; a false alarm just means one more test).
- **Fraud detection** → often high **recall** first (catch the fraud), then improve precision so you don't annoy honest users.
- **Balanced problem** → use **F1** to keep both healthy at once.

The pro mindset: don't ask "what's my accuracy?" Ask "**what does a mistake cost here — a miss or a false alarm?**" — then optimise precision or recall to match. That single question is what real ML engineering is about.

Recap — the 20-second version

- **Accuracy lies** when the important class is rare (99% accurate, 0 caught).
- The **confusion matrix** shows 4 outcomes: hits, false alarms, and misses.
- **Precision** = when you say "yes", how often you're right (avoids false alarms).
- **Recall** = of all real cases, how many you caught (avoids dangerous misses).
- They **trade off** — use **F1** to balance, and choose based on what a mistake costs.

Next up — Video 18: The ML Project Workflow, end to end. You now have algorithms, tuning, and metrics. Next we put it ALL together — the real step-by-step journey from raw data to a working, deployed model, the way it actually happens on the job. See you Day 18.

